

一、AI 交互日志

作业以“城市出租车出行数据分析与智能问答系统”为主题。项目主要包括数据处理、分析可视化、预测模型以及问答接口四个模块。在开发过程中，我结合了 ChatGPT、DeepSeek 等 AI 工具辅助完成代码设计与优化，同时进行了人工理解、修改与验证。

1. 按模块记录关键 Prompt、AI 输出摘要及你的采用/修改/拒绝决策

①

在完成“车费影响因素分析”时，我最初向 AI 提问：如何绘制出租车行程距离与车费关系的散点图？

AI 给出的代码如下：

```
plt.scatter(  
    df['trip_distance'],  
    df['fare_amount']  
)
```

一开始我直接采用了这段代码，但运行后发现出现了明显问题：程序运行速度慢；图像几乎糊成一片；保存图片时间很长。

后来我发现，问题的根源在于数据量过大。作业使用总数据量接近 300 万条，如果直接对全部数据绘制散点图，会产生大量重叠点，不仅影响可视化效果，还会严重拖慢绘图速度。

因此，我对代码进行了优化，采用“随机采样”的方式：

```
sample_df = df.sample(10000)
```

```
plt.scatter(  
    sample_df['trip_distance'],  
    sample_df['fare_amount'],  
    alpha=0.3  
)
```

②

在完成 M2 自选分析时，我最初想到的方案是：分析不同时间段的平均行程距离变化

于是我向 AI 提问：如何分析一天中不同时间段的平均行程距离？

AI 给出的思路是：

```
distance_by_hour = df.groupby('hour')[  
    'trip_distance'
```

].mean())并建议绘制折线图观察趋势。

一开始我使用这个方案，后来观察图像后，我发现这个图像与车费影响因素分析高度重复，因为出租车收费本身主要与行程距离相关。

所以 AI 提出了平均速度变化分析，反映不同时段交通效率变化

```
df['speed'] = df['trip_distance'] / (  
    df['duration'] / 60  
)  
  
speed_by_hour = df.groupby('hour')[  
    'speed'  
].mean()
```

2. 重点记录 AI 犯错的案例（错在哪、如何发现、如何修正）

①

在使用 TensorFlow 构建出租车需求预测模型时，最初直接将区域编号 PULocationID 输入神经网络进行训练。

AI 初始生成的代码如下：

```
X=demand_df[['PULocationID', 'hour', 'weekday']]
```

随后直接进行模型训练，没有对区域编号进行编码处理。

错误原因：

PULocationID 本质上属于类别变量，而不是连续数值变量。

直接输入神经网络后，模型会错误地认为不同区域之间存在数值大小关系，但实际上区域编号只是标签，不具有数学意义。

发现原因

在训练过程中发现：预测结果波动较大、模型拟合效果不稳定

进一步检查特征后，发现区域编号未进行类别编码。

修复过程：

后续采用 One-Hot 编码方式处理：

```
X = pd.get_dummies(  
X,  
columns=['PULocationID']  
)
```

将区域编号转换为多个 0/1 特征，使模型能够正确识别不同区域。

`X = X.astype('float32')`解决 TensorFlow 输入类型兼容问题。

②

在问答系统中，需要从用户输入中提取时间数字，系统采用正则表达式提取数字。

AI 初始建议代码：

`hour = int(re.findall(r'\d+', question)[0])`用于获取用户输入中的小时数字。

错误原因

该写法默认用户输入中一定存在数字

但如果用户未输入数字

`re.findall()` 返回空列表：`[]`

继续访问 `[0]` 会导致：

`IndexError: list index out of range`

发现原因：

在实际运行问答系统时，当输入不包含数字的问题后，程序直接报错并终止。

通过打印：

```
print(re.findall(r'\d+', question))
```

发现部分输入无法匹配数字。

修正过程后续增加列表长度判断：

```
nums = re.findall(r'\d+', question)
```

```
if len(nums) == 0:
```

```
    print('请输入小时，例如：10 点订单量')
```

```
else:
```

```
    hour = int(nums[0])
```

通过异常情况处理，避免程序因用户输入不规范而崩溃。

二、三阶段对比

M2 车费影响因素分析中：

Native 版：

```
df['trip_distance']
```

```
df['fare_amount']

plt.scatter(
    df['trip_distance'],
    df['fare_amount']
)

plt.xlabel('Trip Distance')
plt.ylabel('Fare Amount')
```

问题：数据量过多，异常值影响（极远距离、极高车费）

Prompt 版：

```
plt.scatter(
    df['trip_distance'],
    df['fare_amount'],
    alpha=0.3
)
```

建议：使用透明度；设置图像大小；添加标题。

图像明显更规范，但未主动处理异常值

Vibe 版：

发现图像不清晰后我向 AI 提出问题，后 AI 建议使用随机采样

```
sample_df = df.sample(10000)
```

同时设计

```
plt.scatter(
    sample_df['trip_distance'],
    sample_df['fare_amount'],
    alpha=0.3,
    s=5
)
```

后来在询问 AI，提出图像更加准确改为：

```
plt.scatter(
    sample_df['trip_distance'],
    sample_df['fare_amount'],
    alpha=0.3,
    s=3
)
```

)

通过这个功能的三阶段对比，我的感受是：

AI 很擅长生成代码、翻译自然语言，但不一定真正理解分析意义。

比如在数据分析任务中将图画出来，但未考虑理解变量关系、考虑数据规模

因此，人仍然需要发现 AI 局限，设计分析目标，优化整体分析逻辑。

三、反思

完成这次课程作业后，我对 AI 编程工具的能力边界有了更加深刻的认识。最开始使用 AI 时，我会觉得它像一个万能代码生成器，很多代码只需要一句 Prompt 就能够快速生成。例如数据读取、matplotlib 可视化、TensorFlow 神经网络、随机森林模型等内容，AI 都能马上给出完整代码，效率确实提高了很多。尤其是在一些重复性较强、模板化明显的任务中，AI 的优势非常明显，比如生成训练模型的基本框架、绘制图表、封装函数等，大大减少了基础编码时间，也让我能够把更多精力放在整体项目设计上。

但随着项目不断深入，我逐渐发现 AI 并不真正理解项目本身。它更像是在大量已有代码模式基础上进行翻译，因此很多时候它生成的代码虽然语法正确，但并不一定适合真实场景。

因此，我认为未来最重要的能力，不只是会写代码，而是能够理解问题、正确使用 AI，并判断 AI 生成结果是否真正合理。